HotStuff 패밀리

📋 패밀리 개요

HotStuff는 2019년 VMware Research에서 제안한 **선형 메시지 복잡도(O(n))를 가진 BFT 합의 알고리즘**입니다. PBFT와 Tendermint의 $O(n^2)$ 복잡도를 O(n)으로 개선하여 확장성을 크게 향상시켰습니다.

패밀리 구성 (6가지)

1. **Basic HotStuff**: 원본 3-phase 프로토콜
2. **Chained HotStuff**: 파이프라인 + 3-chain rule
3. **Fast-HotStuff**: 2-phase 최적화
4. **Event-driven HotStuff**: 이벤트 기반 + 2-chain rule
5. **Streamlet**: 간소화된 블록체인 프로토콜
6. **Jolteon**: Responsive BFT + 3가지 경로

---

1. Basic HotStuff

📋 개요

Basic HotStuff는 **리더 수집 방식**으로 O(n) 메시지 복잡도를 달성합니다.

핵심 아이디어: Threshold Signature

기존 (PBFT/Tendermint):

```
All-to-All 투표:
Node 0 → All: VOTE
Node 1 → All: VOTE
Node 2 → All: VOTE
```

```
Node 3 → All: VOTE
= 4 × 3 = 12 messages (O(n²))
```

HotStuff:

```
Leader 수집:
Node 1 → Leader: VOTE
Node 2 → Leader: VOTE
Node 3 → Leader: VOTE
Leader → All: QC (Quorum Certificate)
= 3 + 3 = 6 messages (O(n))
```

QC (Quorum Certificate)

```go
type QC struct {
    blockHash  string       // 블록 해시
    signatures []Signature  // 2f+1 서명들
    round      int         // 라운드 번호
}
```

의미: 2f+1 노드가 이 블록에 투표했다는 증명

3-Phase 프로토콜

```
Leader              Replica1            Replica2            Replica3
  |                    |                   |                   |
  |---PREPARE---------->|                   |                   |
  |---PREPARE---------->|------------------>|                   |
  |---PREPARE---------->|------------------>|------------------>|
  |                    |                   |                   |
  |<--VOTE-PREPARE------|                   |                   |
  |<--VOTE-PREPARE------|<------------------|                   |
  |<--VOTE-PREPARE------|<------------------|<------------------|
  |                    |                   |                   |
  |---PRECOMMIT+QC----->|                   |                   |
  |---PRECOMMIT+QC----->|------------------>|                   |
  |---PRECOMMIT+QC----->|------------------>|------------------>|
  |                    |                   |                   |
  |<--VOTE-PRECOMMIT----|                   |                   |
  |<--VOTE-PRECOMMIT----|<------------------|                   |
  |<--VOTE-PRECOMMIT----|<------------------|<------------------|
  |                    |                   |                   |
```

```
|---COMMIT+QC-------->|                   |                   |
|---COMMIT+QC-------->|------------------>|                   |
|---COMMIT+QC-------->|------------------>|------------------>|
|                     |                   |                   |
|<--VOTE-COMMIT-------|                   |                   |
|<--VOTE-COMMIT-------|<------------------|                   |
|<--VOTE-COMMIT-------|<------------------|<------------------|
|                     |                   |                   |
|---DECIDE+QC-------->|                   |                   |
|---DECIDE+QC-------->|------------------>|                   |
|---DECIDE+QC-------->|------------------>|------------------>|
```

Phase 상세

Phase 1: Prepare

```
Leader:
  1. 블록 생성
  2. PREPARE(block) 전송
  3. 2f+1 VOTE 수집
  4. prepareQC 생성

Replica:
  1. PREPARE 수신
  2. 블록 검증
  3. VOTE(blockHash) → Leader
```

Phase 2: Precommit

```
Leader:
  1. PRECOMMIT(block, prepareQC) 전송
  2. 2f+1 VOTE 수집
  3. precommitQC 생성

Replica:
  1. PRECOMMIT + prepareQC 수신
  2. prepareQC 검증 (2f+1 서명)
  3. VOTE(blockHash) → Leader
```

Phase 3: Commit

```
Leader:
  1. COMMIT(block, precommitQC) 전송
```

```
  2. 2f+1 VOTE 수집
  3. commitQC 생성

Replica:
  1. COMMIT + precommitQC 수신
  2. precommitQC 검증
  3. VOTE(blockHash) → Leader
```

Phase 4: Decide

```
Leader:
  1. DECIDE(commitQC) 전송

Replica:
  1. DECIDE + commitQC 수신
  2. 블록 실행!
```

메시지 복잡도

4노드 예제 (n=4):

- Prepare: 3 (Leader→Replicas) + 3 (Votes) = 6
- Precommit: 3 + 3 = 6
- Commit: 3 + 3 = 6
- Decide: 3 = 3
- **총**: 21 messages = **O(n)**

코드 위치

```
internal/algorithms/hotstuff/hotstuff.go
```

---

2. Chained HotStuff

📋 개요

Chained HotStuff는 **파이프라인 방식**으로 여러 블록을 동시에 처리하여 처리량을 높입니다. Libra (Diem)에서 사용되었습니다.

핵심 아이디어: Generic 메시지 + 3-chain rule

Basic HotStuff 문제:

```
Block 1: Prepare → Precommit → Commit → Decide (4 phases)
Block 2: Prepare → Precommit → Commit → Decide (4 phases)
= Sequential, no pipelining
```

Chained HotStuff 해결:

```
Block 1: Vote
Block 2: Vote (Block 1's Prepare)
Block 3: Vote (Block 2's Precommit, Block 1's Commit)
Block 4: Vote (Block 3's Decide, Block 2's Commit, Block 1's EXECUTE!)
= Pipelined, overlapping phases
```

블록 체인 구조

```
┌──────────────┐     ┌──────────────┐     ┌──────────────┐     ┌──────────────┐
|   Block 1    |────▶|   Block 2    |────▶|   Block 3    |────▶|   Block 4    |
|  (Prepare)   |     | (Precommit)  |     |  (Commit)    |     |  (Decide)    |
|              |     |              |     |              |     |              |
|   parent:    |     |   parent:    |     |   parent:    |     |   parent:    |
|   Block 0    |     |   Block 1    |     |   Block 2    |     |   Block 3    |
|              |     |              |     |              |     |              |
|   QC(B0)     |     |   QC(B1)     |     |   QC(B2)     |     |   QC(B3)     |
└──────────────┘     └──────────────┘     └──────────────┘     └──────────────┘
```

3-Chain Rule

```
if block.parent.parent.parent is justified {
    // 3 연속 블록!
    commit(block.parent.parent)
}
```

Justified: 블록이 2f+1 투표 (QC) 받음

동작 과정

```
Height 1:
  Leader proposes Block 1 (parent: Genesis, QC(Genesis))
  Replicas vote → QC(B1)
  [B1 is justified]

Height 2:
  Leader proposes Block 2 (parent: B1, QC(B1))
  Replicas vote → QC(B2)
  [B2 is justified]

Height 3:
  Leader proposes Block 3 (parent: B2, QC(B2))
  Replicas vote → QC(B3)
  [B3 is justified]
  [3-chain: Genesis ← B1 ← B2 ← B3]
  → Commit B1! ✓

Height 4:
  Leader proposes Block 4 (parent: B3, QC(B3))
  [3-chain: B1 ← B2 ← B3 ← B4]
  → Commit B2! ✓
```

암묵적 투표 (Implicit Voting)

```
Block 3 투표 = {
    Block 3에 대한 투표 (Prepare)
    + Block 2에 대한 암묵적 Precommit
    + Block 1에 대한 암묵적 Commit
}
```

메시지 구조

```go
type ChainedBlock struct {
    height   int
    parent   *ChainedBlock
    parentQC *QC           // 부모에 대한 2f+1 투표 증명
    data     []Transaction
}

type Vote struct {
    blockHash string
```

```
    nodeID    int
}
```

장점

✅ **파이프라이닝**: 높은 처리량
✅ **단순화**: Generic 메시지 하나로 통합
✅ **효율성**: 각 블록당 1 round만 필요

코드 위치

```
internal/algorithms/hotstuff_chained/hotstuff_chained.go
```

---

3. Fast-HotStuff

📋 개요

Fast-HotStuff는 **Optimistic 경로**를 추가하여 정상 상황에서 2-phase로 합의를 달성합니다.

핵심 아이디어: Fast Path vs Regular Path

Fast Path (모든 노드 동의):

```
Prepare → Commit (2 phases)
조건: n votes (100% 동의)
```

Regular Path (일부 노드 불일치):

```
Prepare → Precommit → Commit (3 phases)
조건: 2f+1 votes
```

동작 과정

```
Leader                Replica1 (agree)    Replica2 (agree)    Replica3 (agree)
  |                       |                   |                   |
```

```
  |---PREPARE---------->|                   |                   |
  |---PREPARE---------->|------------------>|                   |
  |---PREPARE---------->|------------------>|------------------>|
  |                     |                   |                   |
  |<--VOTE(agree)-------|                   |                   |
  |<--VOTE(agree)-------|<------------------|                   |
  |<--VOTE(agree)-------|<------------------|<------------------|
  |                     |                   |                   |
  | Check: n votes? YES! → Fast path!                          |
  |                     |                   |                   |
  |---COMMIT+QC-------->|                   |                   |
  |---COMMIT+QC-------->|------------------>|                   |
  |---COMMIT+QC-------->|------------------>|------------------>|
  |                     |                   |                   |
  | SKIP Precommit! → Execute directly                         |
```

일부 노드 불일치 시:

```
Leader              Replica1 (agree)    Replica2 (disagree) Replica3 (agree)
  |                     |                   |                   |
  |---PREPARE---------->|                   |                   |
  |                     |                   |                   |
  |<--VOTE(agree)-------|                   |                   |
  |<--VOTE(disagree)----|<------------------|                   |
  |<--VOTE(agree)-------|<------------------|<------------------|
  |                     |                   |                   |
  | Check: n votes? NO (only 3/4) → Regular path                |
  |                     |                   |                   |
  |---PRECOMMIT+QC----->|                   |                   |
  |      (continue with 3-phase...)                             |
```

Fast Path 조건

```go
func (node *FastHotStuffNode) checkFastPath(votes map[int]bool) bool {
    // All nodes must vote
    if len(votes) == node.numNodes {
        // All votes must be for the same block
        return node.allAgree(votes)
    }
    return false
}
```

블록 해시 비교

```go
// Replica가 투표할 때 블록 해시 포함
type Vote struct {
    blockHash string  // 제안된 블록의 해시
    agree     bool    // 동의 여부
}

// Leader가 확인
func (leader *Node) allAgree(votes []Vote) bool {
    firstHash := votes[0].blockHash
    for _, vote := range votes {
        if vote.blockHash != firstHash {
            return false  // 불일치 발견
        }
    }
    return true
}
```

성능 향상

Best case (Fast path):

- 2 phases (33% 감소)
- n votes 필요

Worst case (Regular path):

- 3 phases (Basic HotStuff와 동일)
- 2f+1 votes

코드 위치

`internal/algorithms/hotstuff_fast/hotstuff_fast.go`

---

4. Event-driven HotStuff

📋 개요

Event-driven HotStuff는 **이벤트 기반 아키텍처**와 **2-chain rule**을 사용하여 코드 가독성과 효율성을 높입니다.

핵심 아이디어: 이벤트 핸들러 + 2-chain

이벤트 핸들러:

```
OnProposal(block)  // 제안 수신
OnVote(vote)       // 투표 수신
OnQC(qc)          // QC 수신
```

2-chain Rule (3-chain 대신):

```
if block.parent is justified {
    commit(block.parent)
}
```

이벤트 흐름

```
┌──────────────────────────────────────────────────────────┐
│                    Event Loop                            │
├──────────────────────────────────────────────────────────┤
│                                                          │
│  ┌──────────────┐      ┌──────────────┐              │
│  │ OnProposal   │──▶   │ Validate     │              │
│  │              │      │              │              │
│  │ - Validate   │      │ Safety       │              │
│  │ - Safety     │      │ Check        │              │
│  │ - Vote       │      └──────────────┘              │
│  └──────────────┘             │                       │
│                               ▼                       │
│                        ┌──────────────┐              │
│                        │   OnVote     │              │
│                        │              │              │
│                        │ - Collect    │              │
│                        │ - Form QC    │              │
│                        └──────────────┘              │
│                               │                       │
│                               ▼                       │
│                        ┌──────────────┐              │
│                        │    OnQC      │              │
│                        │              │              │
```

```
│                        │ - 2-chain?   │             │
│                        │ - Commit     │             │
│                        └──────────────┘             │
└─────────────────────────────────────────────────────┘
```

이벤트 핸들러 상세

OnProposal:

```go
func (node *EventDrivenNode) OnProposal(block *Block) {
    // 1. 안전성 검증
    if !node.safetyPredicate(block) {
        return  // 거부
    }

    // 2. 블록 저장
    node.blocks[block.Hash()] = block

    // 3. 투표
    vote := node.createVote(block)
    node.sendToLeader(vote)
}
```

Safety Predicate:

```go
func (node *EventDrivenNode) safetyPredicate(block *Block) bool {
    // 부모가 justify되어 있거나
    // 또는 내 lockedQC보다 높은 QC를 가짐
    return block.parentQC.Round >= node.lockedQC.Round
}
```

OnVote:

```go
func (leader *EventDrivenNode) OnVote(vote *Vote) {
    // 1. 투표 수집
    leader.votes[vote.BlockHash][vote.NodeID] = true

    // 2. Quorum 확인
    if len(leader.votes[vote.BlockHash]) >= leader.quorum {
        // 3. QC 생성
        qc := leader.createQC(vote.BlockHash)

        // 4. OnQC 이벤트 발생
```

```
        leader.OnQC(qc)
    }
}
```

OnQC:

```go
func (node *EventDrivenNode) OnQC(qc *QC) {
    // 1. QC 저장
    node.highQC = qc

    // 2. 2-chain 확인
    block := node.blocks[qc.BlockHash]
    if block.parent.justified {
        // 2-chain! 부모 커밋
        node.commit(block.parent)
    }

    // 3. 다음 블록 제안 (리더라면)
    if node.isLeader() {
        nextBlock := node.createBlock(block, qc)
        node.OnProposal(nextBlock)
    }
}
```

2-Chain vs 3-Chain

3-Chain (Chained HotStuff):

```
B1 ← B2 ← B3 ← B4
justified  justified  justified

→ Commit B1 (3-chain)
```

2-Chain (Event-driven):

```
B1 ← B2 ← B3
justified  justified

→ Commit B2 (2-chain)
```

차이점:

- 2-chain: 더 빠른 finality

- 3-chain: 더 보수적

장점

✅ **명확한 로직**: 이벤트 기반으로 가독성 높음
✅ **빠른 finality**: 2-chain rule
✅ **확장성**: 이벤트 핸들러 쉽게 추가 가능

코드 위치

```
internal/algorithms/hotstuff_event/hotstuff_event.go
```

---

5. Streamlet

📋 개요

Streamlet은 **교육용으로 설계된 초간단 블록체인 합의 프로토콜**입니다. HotStuff의 핵심 아이디어를 최소화한 버전입니다.

핵심 아이디어: Epoch + Notarization

Epoch: 시간을 epoch으로 나눔
Notarization: 2f+1 투표 = 블록 공증

동작 과정

```
Epoch 0:
  Leader 0: Propose Block A
  Validators vote → Block A notarized ✓

Epoch 1:
  Leader 1: Propose Block B (parent: A)
  Validators vote → Block B notarized ✓

Epoch 2:
```

```
Leader 2: Propose Block C (parent: B)
Validators vote → Block C notarized ✓

3 consecutive epochs! → Finalize Block A!
```

Notarization

```go
type NotarizedBlock struct {
    block *Block
    votes map[int]bool  // 2f+1 votes
}

func (node *StreamletNode) notarize(block *Block) bool {
    return len(node.votes[block.Hash()]) >= node.quorum
}
```

3 Consecutive Epochs Rule

```
┌─────────┐      ┌─────────┐      ┌─────────┐
| Epoch0 |──────▶| Epoch1 |──────▶| Epoch2 |
|Block A |      |Block B |      |Block C |
|(parent:|      |(parent:|      |(parent:|
|  Gen)  |      |   A)   |      |   B)   |
└─────────┘      └─────────┘      └─────────┘
    ✓              ✓              ✓
notarized      notarized      notarized

→ 3 consecutive notarized blocks!
→ Finalize Block A!
```

Longest Notarized Chain

```
          ┌──────┐
          |  B1 | (not notarized)
          └──────┘
             ↑
┌──────┐  ┌──────┐  ┌──────┐
|  A  |→ |  B  |→ |  C  |
└──────┘  └──────┘  └──────┘
  ✓        ✓        ✓
```

```
Chain 1: A → B → C (length 3, all notarized)
Chain 2: A → B → B1 (length 3, B1 not notarized)

→ Follow Chain 1!
```

리더 선출

```go
func (node *StreamletNode) getLeader(epoch int) int {
    return epoch % node.numNodes
}
```

Round-robin, 매우 간단!

코드 위치

```
internal/algorithms/streamlet/streamlet.go
```

---

6. Jolteon (Responsive BFT)

📋 개요

Jolteon은 **3가지 합의 경로**를 지원하는 Responsive BFT입니다. 네트워크 상태에 따라 최적의 경로를 자동 선택합니다.

핵심 아이디어: 3가지 Threshold

```
┌──────────────────────────────────────┐
│     Optimistic Path (n votes)        │  ← 모든 노드 동의
│              ↓                       │
│     Responsive Path (n-f votes)      │  ← 정직한 노드만
│              ↓                       │
│     Fallback Path (2f+1 votes)       │  ← 전통적 BFT
└──────────────────────────────────────┘
```

Threshold 계산 (n=4, f=1)

```
optimisticThreshold := n           // 4 votes (100%)
responsiveThreshold := n - f       // 3 votes (75%)
fallbackThreshold   := 2*f + 1     // 3 votes (75%)
```

경로 선택

```go
func (node *JolteonNode) selectPath(voteCount int) Path {
    if voteCount >= node.optimisticThreshold {
        return OptimisticPath  // n votes
    } else if voteCount >= node.responsiveThreshold {
        return ResponsivePath  // n-f votes
    } else if voteCount >= node.fallbackThreshold {
        return FallbackPath    // 2f+1 votes
    }
    return NoPath
}
```

동작 예제

시나리오 1: 모든 노드 빠르게 응답 (Optimistic)

```
Leader proposes Block A
All 4 nodes vote quickly
→ 4/4 votes (n)
→ Optimistic Path! 즉시 커밋 ✓
```

시나리오 2: 한 노드 느림 (Responsive)

```
Leader proposes Block A
3 nodes vote quickly, 1 node slow
→ 3/4 votes (n-f)
→ Responsive Path! 빠르게 커밋 ✓
(느린 노드 기다리지 않음)
```

시나리오 3: 네트워크 문제 (Fallback)

```
Leader proposes Block A
Only 3 nodes respond
→ 3/4 votes (2f+1)
→ Fallback Path (2-phase)
→ 2단계 후 커밋
```

Responsive의 의미

전통적 BFT:

- 항상 2f+1 대기
- 느린 f개 노드 때문에 지연

Responsive BFT:

- n-f 투표면 충분
- 정직한 노드만으로 빠른 합의
- 느린 노드 무시 가능

Phase 별 처리

```go
func (node *JolteonNode) handleVotes(blockHash string) {
    voteCount := len(node.votes[blockHash])

    switch node.selectPath(voteCount) {
    case OptimisticPath:
        // 1-phase commit!
        node.commit(blockHash)

    case ResponsivePath:
        // 1-phase commit!
        node.commit(blockHash)

    case FallbackPath:
        // 2-phase commit
        node.enterPrecommit(blockHash)
    }
}
```

비교: Fast-HotStuff vs Jolteon

특징	Fast-HotStuff	Jolteon
경로 수	2 (Fast, Regular)	3 (Optimistic, Responsive, Fallback)
Fast 조건	n votes	n votes

특징	Fast-HotStuff	Jolteon
중간 경로	❌	✅ n-f votes
Responsive	❌	✅

코드 위치

```
internal/algorithms/jolteon/jolteon.go
```

📊 HotStuff 패밀리 비교

알고리즘	Phase 수	메시지 복잡도	특징	Finality
Basic	3-4	O(n)	원본, QC 기반	4 phases
Chained	1 (per block)	O(n)	파이프라인, 3-chain	3 blocks
Fast	2-3	O(n)	Optimistic	2-3 phases
Event-driven	3	O(n)	이벤트 기반, 2-chain	2 blocks
Streamlet	1	O(n)	초간단, epoch	3 epochs
Jolteon	1-2	O(n)	3경로, responsive	1-2 phases

🎯 선택 가이드

교육/학습:

- Streamlet: 가장 간단

프로덕션:

- Chained HotStuff: 높은 처리량
- Jolteon: 다양한 네트워크 환경

최적화 우선:

- Fast-HotStuff: 정상 시 빠름
- Jolteon: 적응형

📚 참고 자료

- [HotStuff 논문] Yin et al. (2019)
- [Chained HotStuff] Libra/Diem
- [Streamlet 논문] Chan & Shi (2020)
- [Jolteon 논문] Gelashvili et al. (2022)
- [코드] `internal/algorithms/hotstuff*/`

← Tendermint | Narwhal Family →